Prima Applicazione AI

Boolean Master · Lezione 12 · Lab 4 ore

Dal concetto al codice

Le tre lezioni di AI: dalla teoria al codice Python.

LEZIONE 10

Come funziona un LLM

Completa il contesto, un token alla volta.

→

LEZIONE 11

Prompt engineering

Scrivere le istruzioni, nel playground.

→

LEZIONE 12 · OGGI

Le chiamate API

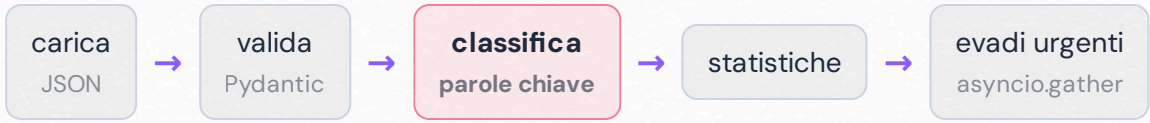
In Python, in un progetto locale.

I ruoli system e user diventano codice, la temperature un parametro. In più: gestione degli errori e streaming.

Il progetto del Modulo 1

Il progetto di riepilogo "AI Help Desk" che avete fatto a fine Modulo 1.

PROGETTO M1 · AI HELP DESK Un programma che legge ticket di supporto da un file JSON (alcuni scritti male), li valida, li classifica per categoria, calcola statistiche ed evade gli urgenti. Tutto in Python, senza AI.



Il passo della classificazione è fatto con regole a parole chiave. Oggi sostituiamo quel passo con un LLM.

I limiti delle parole chiave

Nel progetto lo avevate già visto: le regole a parole chiave non bastano.

COSA SA FARE IL CODICE TRADIZIONALE

Contare, filtrare, ordinare, match esatto di stringhe, pattern fissi (parole chiave, regex).

COSA CI SERVE

Capire il testo: classificare, cogliere tono e urgenza, dedurre quello che non c'è scritto.

`"Non mi è mai arrivato il collo."` **→ nessuna parola chiave → default**

Un lettore umano capisce che è una **spedizione**.

Il problema, dal vivo

DAL PROGETTO M1 **`ClassificatoreParoleChiave`** `tickets.json`

- Lanciamo il classificatore a parole chiave del progetto sui ticket reali
- Sui ticket puliti funziona; su `"il collo che non arriva"` e altri finisce nel default
- Tono, urgenza e duplicati a parole chiave non si ricavano

Domanda: invece di aggiungere altre parole chiave, possiamo chiedere a un modello?

Programma della lezione

Un progetto Python locale, in nove tappe.

01
Come funzionano le API

02
Prepariamo l'ambiente

03
La prima chiamata

04
I parametri

05
La conversazione

06
Gli errori

07
Streaming

08
Strumenti pronti

09
Le due app

1. Come funzionano queste API

Abbonamento, API, endpoint e SDK

Due modi di usare un LLM

ChatGPT e le API sono due prodotti diversi sopra lo stesso modello, con costi separati.



CHATGPT

L'app, a mano

Interfaccia grafica, un abbonamento mensile (~20€/mese). Usi il modello da browser o app.

`chatgpt.com`

API

Dal tuo codice

Il tuo programma chiama il modello via rete e paghi a consumo, a token, senza canone.

`platform.openai.com`

Sono scollegati: puoi usare solo le API, senza l'abbonamento, e farti un assistente personale pagato a consumo.

Gli endpoint REST

Lo avete già visto nel progetto M1: una richiesta HTTP con un JSON.

```
RICHIESTA → POST /v1/responses
Authorization: Bearer <API key>
Content-Type: application/json

{ "model": "...", "input": "..." }
```

```
RISPOSTA → JSON
{ "output": [ ... ],
  "usage": { "input_tokens": 19,
    "output_tokens": 31 } }
```

Nel progetto M1 la chiamata di assaggio era un `httpx.post` su `chat/completions`. Si può fare tutto a mano con httpx.



L’SDK ufficiale

La libreria openai, sviluppata e mantenuta da OpenAI, fa la stessa cosa in modo più leggibile.

HTTPX A MANO

```python
import httpx

r = httpx.post(
    "https://api.openai.com/v1/responses",
    headers={"Authorization": f"Bearer {key}"},
    json={"model": MODEL, "input": testo},
)
print(r.json()["output"][0]["content"][0]["text"])
```

SDK OPENAI

```python
from openai import OpenAI

client = OpenAI()
r = client.responses.create(model=MODEL, input=testo)
print(r.output_text)
```

Nasconde i dettagli HTTP, gestisce retry, errori ed eventi di streaming.

Responses API · la nuova

Chat Completions · la precedente



2. Prepariamo l’ambiente

Un progetto Python locale, da zero



Dal notebook al progetto locale

Nel Modulo 1 avete usato i notebook. Oggi un progetto locale, con file .py.

NOTEBOOK · MODULO 1

A celle, l'output subito sotto. Comodo per studiare.

→

PROGETTO LOCALE · OGGI

File `.py`, un ambiente isolato, librerie installate.

Il venv lo avete già incontrato nei notebook. Oggi lo facciamo sulla vostra macchina, così sapete partire un progetto da soli.



Requisiti

Per lavorare a un progetto Python in locale servono tre cose.

01

Python

Il linguaggio e l'interprete che esegue i nostri file `.py`.

02

Ambiente virtuale (venv)

Una cartella isolata con le librerie di questo progetto.

03

pip

Lo strumento per installare le librerie (qui: la libreria openai).

Gli strumenti

Prima un'overview di cosa serve, poi ci assicuriamo che tutti li abbiano e funzionino.

1. Python

Il linguaggio e l'interprete che esegue i file .py. Probabilmente lo avete già.

```
python --version            # verifica la versione (serve la 3.x)
python primo_script.py      # esegue uno script
```

Installazione

Da python.org (spunta "Add Python to PATH"). Su Mac e Linux spesso c'è già come python3.

PATH

"Add to PATH" fa sì che il comando python si possa richiamare da qualsiasi cartella del terminale.



2. L’ambiente virtuale (venv)

Una cartella isolata con il suo Python e le sue librerie, separata dagli altri progetti.

1. Crea
l'ambiente del progetto

```
python -m venv .venv
```

2. Attiva
entri nell'ambiente

```
source .venv/bin/activate
```

È attivo? Vedi `(.venv)` davanti al prompt.

Per uscire: `deactivate`

I comandi di creazione e attivazione cambiano un po' per sistema operativo: li vediamo tra poco.

3. pip: installare le librerie

Con il venv attivo, pip installa le librerie dentro l'ambiente del progetto.

```
pip install openai                # installa una libreria
pip install -r requirements.txt   # installa tutte quelle elencate nel file
pip list                          # mostra cosa è installato nel venv
```

`requirements.txt` è il file con l'elenco delle librerie del progetto e le loro versioni.



Setup per tutti

Arriviamo al punto: alla fine tutti avranno un ambiente locale pronto all'uso.



I quattro ambienti

Ora facciamo il setup insieme: ognuno sul proprio sistema, finché tutti arrivano in fondo.



Windows

PowerShell



WSL

Ubuntu dentro Windows · `consigliato`



Linux

Ubuntu/Debian



macOS

zsh

Su Windows conviene WSL: è un Ubuntu integrato, e i comandi sono gli stessi di Linux e Mac.

Setup: Windows (PowerShell)

Installa Python da python.org spuntando "Add python.exe to PATH".

```
# apri il terminale: cerca "PowerShell" nel menu Start
python --version                # verifica che Python sia installato
py -3.12 -m venv .venv          # crea l'ambiente virtuale
.\.venv\Scripts\Activate.ps1    # attivalo (apparirà (.venv) nel prompt)
pip install -r requirements.txt # installa le librerie
```

Se l'attivazione è bloccata ("running scripts is disabled"), sblocca gli script e riprova:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```



Setup: WSL (Ubuntu su Windows)

La prima volta si installa WSL; poi si apre il terminale Ubuntu e si lavora come su Linux.

```
# una volta sola, in PowerShell come amministratore, poi riavvia il PC:
wsl --install                    # installa WSL e Ubuntu
```

```
# apri il terminale Ubuntu: lancia "wsl" (o apri l'app "Ubuntu" dal menu Start)
wsl
python3 --version                # verifica che ci sia Python
sudo apt update                  # aggiorna l'elenco dei pacchetti
sudo apt install -y python3-venv python3-pip   # installa venv e pip
python3 -m venv .venv            # crea l'ambiente
source .venv/bin/activate        # attivalo
pip install -r requirements.txt  # installa le librerie
```



Setup: Linux (Ubuntu/Debian)

Apri il terminale (su Ubuntu: Ctrl+Alt+T).

```
python3 --version                  # verifica che ci sia Python
sudo apt update                    # aggiorna l'elenco dei pacchetti
sudo apt install -y python3-venv python3-pip   # installa venv e pip
python3 -m venv .venv              # crea l'ambiente
source .venv/bin/activate          # attivalo
pip install -r requirements.txt    # installa le librerie
```

Su altre distribuzioni cambia solo il gestore pacchetti (es. `dnf` su Fedora).



Setup: macOS

Apri il Terminale (Cmd+Spazio, scrivi "Terminale"). Installa Python da python.org.

```
python3 --version                # verifica Python (non quello di sistema)
python3 -m venv .venv            # crea l'ambiente
source .venv/bin/activate        # attivalo
pip install -r requirements.txt  # installa le librerie
```

In alternativa installa Python con Homebrew: `brew install python`.



Le librerie del progetto

requirements.txt elenca le librerie con la versione esatta: a tutti gira la stessa cosa.

```
openai==2.42.0
pydantic==2.13.4
python-dotenv==1.1.1
```

`pip install -r requirements.txt` le installa nel venv.

Con queste installate nel venv, l'ambiente locale è pronto.



Fallback: Google Colab

Chi ha problemi col setup locale non resta indietro.

LOCALE · CONSIGLIATO

venv + script .py

Il modo dei progetti veri. È il filo principale di oggi.

COLAB · FALLBACK

Apri nel browser

Zero installazioni, solo un account Google. File > Carica blocco note.

Notebook fallback: `prima_applicazione_ai_colab.ipynb`.

Setup: tutti insieme

TUTTI INSIEME **l'ambiente del primo progetto Python**

- ☐ `python --version`
- ☐ venv creato e attivo
- ☐ `pip install -r requirements.txt`

La chiave nel `.env` la vediamo appena passiamo al codice. Chi si blocca usa Colab.



Ora iniziamo con il codice

Ambiente pronto: da qui in poi scriviamo, dal vivo e da zero.



Pausa

Variabili di ambiente

La chiave è un segreto: sta in un file .env, fuori dal codice, e la leggiamo come variabile d'ambiente.

crea il file .env (non va su Git)

```
OPENAI_API_KEY=sk-...
```

`cp .env.example .env`, poi incolla la chiave. È nel `.gitignore`: non finisce su Git.

nel codice la leggi così

```
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()          # legge il .env
client = OpenAI()      # usa OPENAI_API_KEY
```

Regola: la chiave non si scrive mai nel codice e non si committa. Conviene anche un tetto di spesa sull'account.



3. La prima chiamata

responses.create, e cosa torna indietro



responses.create

Crei il client e lo chiami: gli dici quale modello e cosa vuoi.

```python
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI()

risposta = client.responses.create(
    model="gpt-4.1-mini",
    instructions="Sei un classificatore di ticket. Rispondi con una parola.",  # il system prompt
    input="Non mi è mai arrivato il collo.",                                    # la richiesta
)
```

`instructions` sono le istruzioni di sistema (il system prompt di giovedì), `input` è cosa chiediamo.

La struttura della response

È un oggetto JSON con molti campi. Quelli utili da conoscere:


```
{
  "id": "resp_0acf ... ",
  "model": "gpt-4.1-mini-2025-04-14",
  "status": "completed",
  "output": [
    { "role": "assistant",
      "content": [{ "type": "output_text",
        "text": "Il vuoto dello spazio ... " }] }
  ],
  "usage": { "input_tokens": 19,
             "output_tokens": 31,
             "total_tokens": 50 },
  "temperature": 1.0,
  "max_output_tokens": null,
  "store": true,
  "text": { "verbosity": "medium" },
  "tools": []
}
```


- `output` il testo generato
- `usage` i token usati, cioè il costo
- `model` il modello esatto
- `status` se è andata a buon fine
- `id` per concatenare le risposte
- `temperature, store, text` i parametri usati

Oggi andiamo veloci, ci basta arrivare al dunque: il deep dive sui parametri (e su token e costi) è la prossima lezione.



Classificazione con l'LLM

Il ticket che il classificatore a parole chiave metteva nel default.

PAROLE CHIAVE

"Non mi è mai arrivato il collo."

→ **default**

LLM

"Non mi è mai arrivato il collo."

→ **spedizione**

Il prompt engineering di giovedì vale identico: ora il prompt è una stringa Python.

Passiamo al codice

Chiudiamo le slide e scriviamo lo script da zero.



Script 1

Obiettivo: la prima chiamata a un LLM, e classificare un ticket con una parola.

CREA UN FILE **`01_prima_chiamata.py`**

- La prima `responses.create`, con `instructions` + `input`, e `output_text`
- Classifichiamo il ticket difficile del progetto → spedizione



4. I parametri

La temperature, e le altre opzioni



La temperature

Sullo stesso ticket: a 0 ripetibile, alta varia.

`temperature = 0 · classificazione`

```
spedizione
spedizione
spedizione
```

Stabile e ripetibile: utile per un classificatore.

`temperature = 1.3 · frase di avviso`

```
Verificare lo stato della consegna…
Avviso: collo non ricevuto…
Controllare la tracciabilità…
```

Più varietà: utile quando vuoi creatività.

```
client.responses.create(model="gpt-4.1-mini", input=ticket, temperature=0)
```

Lo stesso schema con Anthropic

Anthropic si chiama in modo simile: lo schema è lo stesso.

OPENAI

```python
from openai import OpenAI

client = OpenAI()
client.responses.create(
    model="gpt-4.1-mini",
    instructions="Sei un classificatore ... ",
    input=ticket,
)
```

ANTHROPIC

```python
from anthropic import Anthropic

client = Anthropic()
client.messages.create(
    model="claude-sonnet-4-6",
    system="Sei un classificatore ... ",
    messages=[{"role": "user", "content": ticket}],
)
```

confronto provider e prezzi → Modulo 2



Passiamo al codice

Chiudiamo le slide e scriviamo lo script da zero.



Script 2

Obiettivo: vedere come temperature cambia le risposte del modello.

CREA UN FILE **02_parametri.py**

- `temperature=0` rilanciata 3 volte → output identico
- `temperature` alta su una frase di avviso → 3 risultati diversi
- Un cenno a `max_output_tokens`



Pausa

5. La conversazione

Il modello non ha memoria: il contesto lo teniamo noi

Il contesto della conversazione

Ogni chiamata è isolata. Per continuare un discorso, passiamo l'intera lista di messaggi.

SOLO L'ULTIMO MESSAGGIO

USER Spiegamela.

ASSISTANT Spiegarti cosa?

Non sa di cosa parli.

CON TUTTA LA LISTA

USER Raccontami una barzelletta.

ASSISTANT Perché il libro di mate era triste? Aveva troppi problemi.

USER Spiegamela.

ASSISTANT È un doppio senso su "problemi"...

IN BREVE Il modello non ricorda: la lista dei messaggi la costruiamo e gliela passiamo noi a ogni chiamata.

previous_response_id

Lo stato della conversazione può tenerlo il server, invece di rimandare la cronologia.

TU PASSI TUTTA LA LISTA

Controllo pieno sul contesto. È la cosa da padroneggiare, e nel chatbot facciamo così.

previous_response_id

Lo stato lo tiene il server: più snello, ma le instructions vanno rimandate e lo stato scade.

Di default il contesto lo gestisci tu costruendo l'input.

Passiamo al codice

Chiudiamo le slide e scriviamo lo script da zero.



Script 3

Obiettivo: dare al modello la memoria della conversazione.

CREA UN FILE `03_conversazione.py`

- `"Spiegamela."` da sola → il bot non sa di cosa parli
- Con la conversazione intera nella lista → spiega la barzelletta giusta
- Il contesto lo costruiamo e glielo passiamo noi



6. Gli errori

Errori tipizzati, retry e backoff



Le eccezioni dell’API

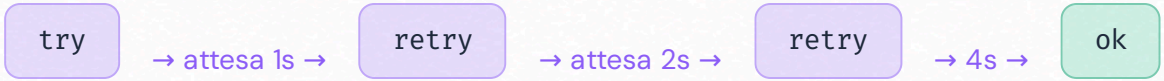
In chat non li vedi; nel codice sì. L'SDK li mappa in eccezioni da catturare con try/except.

chiave sbagliata o assente	`AuthenticationError · 401`
troppe richieste	`RateLimitError · 429`
la chiamata è troppo lenta	`APITimeoutError`
la rete non risponde	`APIConnectionError`
richiesta malformata	`BadRequestError · 400`
errore generico	`APIError (base)`

IN BREVE Un'app vera non assume mai che la chiamata vada a buon fine: ogni errore ha la sua eccezione.

Retry e backoff

Sugli errori temporanei si riprova aspettando un po' di più ogni volta. L'SDK lo fa già.




```python
from openai import OpenAI

client = OpenAI(max_retries=4, timeout=20)
```

Spesso non scrivi il retry a mano: basta sapere che c'è e saperlo regolare.

Passiamo al codice

Chiudiamo le slide e scriviamo lo script da zero.



Script 4

Obiettivo: gestire gli errori dell'API senza far crashare il programma.

CREA UN FILE **04_errori.py**

- Una funzione che avvolge la chiamata in `try/except`
- Un client con una chiave finta → catturiamo `AuthenticationError`
- Un client con `max_retries` e `timeout`

7. Streaming

La risposta pezzo per pezzo



Vantaggi dello streaming

Il testo compare mentre il modello lo genera, invece che tutto alla fine.

SENZA STREAMING

La funzione aspetta e consegna tutto in blocco. Per una risposta lunga, secondi di attesa a schermo fermo.

CON STREAMING

Il testo cresce a pezzi, subito. Non è più veloce in totale, ma l'utente vede la risposta arrivare.

Come si implementa

Con stream=True ricevi un flusso di eventi: prendi il testo dai delta.

```python
from openai import OpenAI

client = OpenAI()
stream = client.responses.create(model="gpt-4.1-mini", input="...", stream=True)

for evento in stream:
    if evento.type == "response.output_text.delta":
        print(evento.delta, end="", flush=True)
```

created → `delta "Lo"` `delta " strea"` `delta "ming"` → completed

Passiamo al codice

Chiudiamo le slide e scriviamo lo script da zero.



Script 5

Obiettivo: far comparire la risposta a pezzi, in tempo reale.

CREA UN FILE **05_streaming.py**

- La stessa chiamata con `stream=True`
- Il loop sugli eventi che stampa i `delta` man mano (`print( ... , end="", flush=True)`)



Pausa

8. Strumenti pronti all’uso

Strumenti integrati nella Responses API



Gli strumenti integrati

La Responses API ha strumenti pronti: si attivano con un parametro, senza scrivere l'integrazione.

`web_search`

Cerca sul web e cita le fonti.

`file_search`

Cerca nei tuoi file.

`code_interpreter`

Esegue codice.

`image generation`

Genera immagini.

`MCP`

Collega strumenti esterni.

Oggi li mostriamo soltanto: il deep dive su tool, modelli, provider e prezzi è già la prossima lezione; i tool vostri (function calling) e il RAG arrivano nel Modulo 3.

9. Le due app

Mettiamo insieme i pezzi



App 1: il chatbot multi-turno

Un loop che mette insieme contesto, streaming e gestione errori.

```
input dell'utente
```

↓

```
append alla lista messaggi
```

↓

```
responses.create(stream=True)
```

↓

```
stampa a pezzi, poi append risposta
```

- contesto · cap 5
- streaming · cap 7
- errori · cap 6

È un chatbot come ChatGPT, scritto da noi, pagato a consumo.



Passiamo al codice

Chiudiamo le slide e scriviamo lo script da zero.



Script 6

Obiettivo: un chatbot multi-turno che ricorda il contesto.

CREA UN FILE **06_chatbot.py**

- Il loop con lista `messaggi` + streaming + `try/except`
- Lo proviamo: "ricordati che mi chiamo Marco" → "come mi chiamo?" → "Ti chiami Marco"



App 2: il classificatore con l'LLM

Sostituiamo il classificatore a parole chiave del progetto con uno che chiede al modello.

PRIMA · PAROLE CHIAVE (M1)

Regole fisse. Si rompe sui ticket scritti in modo non standard.

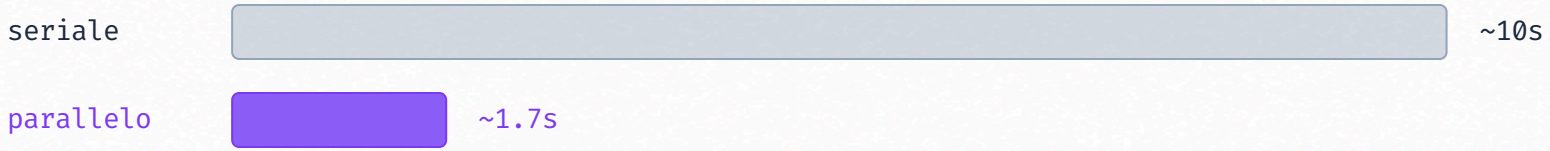
ORA · LLM

Una funzione che chiama il modello (temperature 0). Il resto del programma non cambia.

Riusiamo il modello dati `Ticket` (Pydantic) del vostro progetto per i dati in ingresso.

Classificazione in parallelo

Nel progetto evadevate gli urgenti in parallelo: facciamo lo stesso con le classificazioni.



`AsyncOpenAI` + `asyncio.gather`: lo stesso pattern del lab async, su 6 ticket.

Passiamo al codice

Chiudiamo le slide e scriviamo lo script da zero.



Script 7

Obiettivo: classificare tanti ticket in parallelo.

CREA UN FILE **07_classificatore.py**

- `Ticket` (Pydantic) e una funzione `classifica` che chiama il modello
- Classifichiamo i ticket prima uno alla volta, poi con `asyncio.gather`
- Confrontiamo i tempi: seriale vs parallelo

Nota: in un file `.py` si usa `asyncio.run`; nel notebook Colab si usa `await` diretto.



Ora provate voi

ESERCIZIO 1 · MEDIO

SE AVANZA TEMPO

Il revisore severo.

Un assistente a cui passate una funzione Python e che ne fa la review. Deve:

- segnalare **solo problemi veri** (correttezza, casi limite, sicurezza), niente pignolerie di stile
- se la funzione è a posto, dire chiaramente "nessun problema" e **non inventarne**
- per ogni problema: gravità (alta / media / bassa) e la correzione in una riga

La sfida è il prompt: scrivetelo così bene da farlo comportare esattamente così, anche su una funzione pulita (provatelo su una col bug e una senza).



Ora provate voi

ESERCIZIO 2 · PIÙ TOSTO

SE AVANZA TEMPO

Il triage che sa dire di no.

Un assistente di supporto: gli passate il messaggio di un cliente e lui decide. Deve:

- se può, **rispondere** nel tono dato (cortese e conciso), senza inventare politiche aziendali che non conosce
- **passare a un umano** se il messaggio riguarda rimborsi, dati personali o questioni legali, o se non è sicuro
- essere coerente: lo stesso messaggio deve dare **sempre** la stessa decisione

La sfida è tutta nel prompt: regole di escalation, rifiuto, tono e coerenza. Provatelo su un caso facile e su uno ambiguo.

Riepilogo

Cosa abbiamo fatto oggi.

Dal classificatore a parole chiave del progetto, alla prima chiamata via API, ai parametri, alla response e alla conversazione, alla gestione degli errori, allo streaming, agli strumenti integrati, fino a due app: un chatbot e il classificatore con l'LLM.

`venv + progetto locale` `prima chiamata` `parametri` `conversazione` `errori` `streaming` `web_search` `chatbot + classificatore`

I prossimi moduli

Nel Modulo 2 quello che abbiamo costruito diventa più robusto e strutturato.

LLM APIs DEEP DIVE

OpenAI, Anthropic, Groq a confronto, e i prezzi.

TOKEN & COSTI

Contare e ottimizzare i token con tiktoken.

LANGCHAIN

Strutturare le app invece di scrivere tutto a mano.

STRUCTURED OUTPUT

Vincolare la risposta a uno schema, così il formato è garantito.

Modulo 3 · tool propri e RAG

Kahoot

Verifichiamo insieme: 5 domande sulla lezione di oggi.

Entrate su

kahoot.it

e scrivete il **PIN** che ora vi dico

https://play.kahoot.it/v2/lobby?quizId=5be22b69-12d1-424c-b9f8-633264f60799